

Solving the Subcircuit Extraction Problem by Using Bit-Parallel Filtering Algorithm

Chia Shin Ou, Chin Lung Lu and R. C. T. Lee

Department of Computer Science

National Tsing Hua University

d9662836@oz.nthu.edu.tw, clu@cs.nthu.edu.tw and rctlee@ncnu.edu.tw

Abstract

The subcircuit extraction problem is to find all occurrences of a pattern circuit S in a larger main circuit T . It is an important topic in many areas of VLSI circuit design. In this paper, we propose an algorithm to solve the problem. Our approach transforms both main and pattern circuits into weighted directed graph consisting of positive weights on edges and then finds the all possible candidates of T for the pattern circuits by using the bit-parallel filtering algorithm. We also apply a bit-parallel mechanism to filter out all subcircuits from the surviving main circuit which do not exactly match with S . The remaining circuits found in T will exactly match with S .

1 Introduction

Given two circuits, a smaller pattern circuit S and a larger main circuit T , the *subcircuit extraction problem*[6] is to find all subcircuits in T that is identical to S . It is usual that the main circuit T has a much larger size than the pattern circuit S . A special case of the subcircuit extraction problem where the two circuits T and S have the equal size is known as the Layout-versus-Schematic (LVS) checking in the computer-aided chip design flow. If we represent the circuits as labeled graphs, the LVS problem becomes a special graph isomorphism problem.

The first paper discussing the subcircuit extraction problem was [2] which proposed the SubGemini algorithm. There are two phases in SubGemini algorithm. Phase I identifies all possible locations of the subcircuit in the main circuit and also finds all the possible vertices as candidates in the main circuit. It is considered as a filter. In Phase II, each instance is checked to determine whether it is identical to the subcircuit. Vijaykrishnan presented an approach called SUBGEN [3] to model the subcircuit extraction

problem using the genetic algorithm. It makes use of the fitness function and genetic operators to find the candidate strings with the maximum fitness. Because a genetic algorithm is a heuristics algorithm, SUBGEN cannot guarantee to find all the subcircuits. Thus, it can not guarantee that it will find the optimal solution. And, Zhang and Wunsch proposed a neural networks based Heuristic Dynamic Programming algorithm for subcircuit extraction[4]. They also proposed a fuzzy attributed graph approach[5] with a heuristic searching to solve the subcircuit extraction problem.

The DECIDE algorithm proposed by [1] uses a recursive Breadth-First searching (BFS) approach for the identification algorithm, and also obtained a weight function which is based on the number of neighbors' type for each node. The weight value is used to filter out some impossible nodes in the main circuit. However, DECIDE algorithm would find the erroneous answer in the following case shown in Fig. 1.

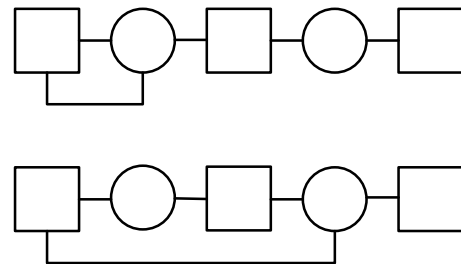


Fig 1. (a) representing the subcircuit S and (b) representing the target circuit G . Each vertex has its weight which is presented by bold word.

To avoid this case, our algorithm obtains an exact match in the final. First, our algorithm represents both circuits T and S by *weighted directed graph* where the weights are on the edges, and each weight on an edge indicates the

connection between two elements in the circuit. For each transistor x in S , we conduct a search in T for all possible candidate of x through a bit-parallel operations. We consider the above as a filter step and obtained the candidate set for each transistor in S . Finally, from candidate set, we obtain a breadth-first search to find out all occurrences of x in T .

2 The Representation of Circuits

We are given a circuit consisting of NMOS and PMOS transistors, and for each transistor, there are drain, gate and source, denoted as d , g and s respectively. For the circuit graph, each transistor is a vertex and each vertex representing a transistor has three terminals, namely drain, gate and source, denoted by d , g and s respectively. An edge linking two terminals is called a link.

A typical example is shown in Fig. 2.

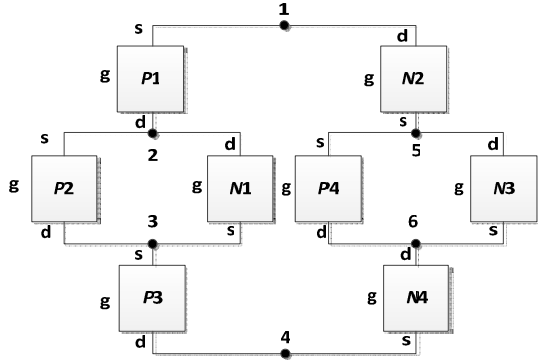


Fig. 2 A circuit graph consisting of PMOS and NMOS transistors represented by square nodes.

P_x and N_x denote the PMOS and NMOS respectively where x is the index of the transistor, and black points denote the terminals.

In our research, we represent a circuit by a weighted directed graph (WDG) $G=(V, E)$ which is a connected graph with weights on edges where V is the vertex set and E is the edge set. In our case, there are two kinds of vertices in V , PMOS and NMOS vertices. Denote $\tau(x)$ to be the transistor type of vertex x , $\tau(x) \in \{\text{PMOS}, \text{NMOS}\}$. And, denote $nmos(x)$ and $pmos(x)$ to be the number of NMOS and PMOS vertices whom vertex x connects to, respectively. For any two vertices u and v , (u, v) denotes the edge from u to v . There are nine types of connections between two vertices.

- Type 1 (bit 1): source to source (ss).
- Type 2 (bit 2): source to gate (sg).
- Type 3 (bit 3): source to drain (sd).
- Type 4 (bit 4): gate to source (gs).
- Type 5 (bit 5): gate to gate (gg).
- Type 6 (bit 6): gate to drain (gd).
- Type 7 (bit 7): drain to source (ds).
- Type 8 (bit 8): drain to gate (dg).
- Type 9 (bit 9): drain to drain (dd).

Let $type(i)$ be the Boolean value defined as follows:

$$type(i) = \begin{cases} 1 & \text{if the link between two MOSs belongs to type } i \\ 0 & \text{if otherwise} \end{cases}$$

Thus every connection of vertices x and y is defined by a bit vector $[ss, sg, sd, gs, gg, gd, ds, dg, dd]$. For example, the connection vector between P_1 and P_2 in Fig. 2 is $[0,0,0,0,0,0,1,0,0]$.

Suppose x and y are labeled as vertices of two transistors in the circuit C . The weight W_C on (x, y) can be computed in the following:

$$W_C(x, y) = \sum_{i=1}^9 type(i) \cdot 2^{i-1} \quad (1)$$

For instance, x is connected to y by sd and ds . Its bit vector is $[0,0,1,0,0,0,1,0,0]$ and $W_C(x, y) = 68$. The WDG of the circuit in Fig. 2 is shown in Fig. 3.

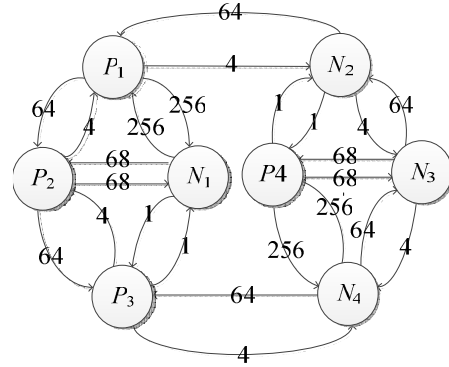


Fig. 3 The WDG of the circuit in Fig. 2 where a circle denotes the vertex corresponding to the transistor.

For every node which is a transistor A , we record the numbers of PMOS and NMOS transistors connected to A . This information is stored in Table 1. Examples are shown in Fig. 4.

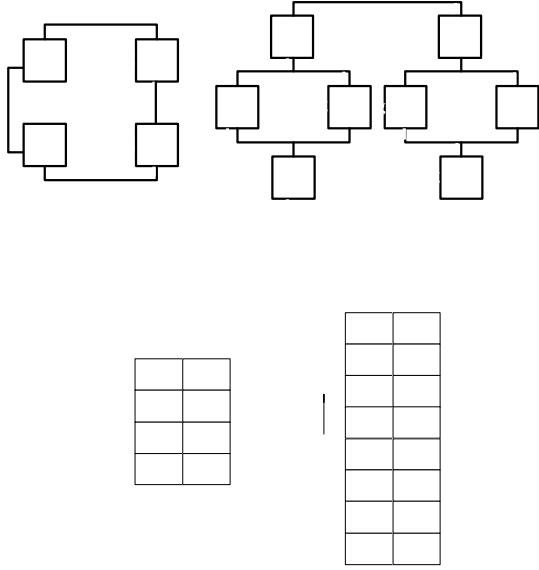


Fig. 4 (a) Circuits S and T (b) The transistor connection number of transistors in S and T

3 The Rules for the Filter Steps

In this paper, transistor-level circuits are considered. We consider MOS-type transistors only. The source and drain terminals of NMOS and PMOS are considered as different terminals and not swappable. Many papers [1,4,5] proposed rules for the filtering step to avoid spending too much time in searching. Powerful and efficient rules for filtering are one of the most important issues for subcircuit extraction problem. Four efficient rules for the filter step are used in our research.

Rule 1: Suppose we consider a transistor x in S and a transistor x' in T . If x' is a candidate for matching x , $pmos(x) \geq pmos(x')$ and $nmos(x) \geq nmos(x')$.

Consider Fig. 4. In this case, P_1' is connected to one PMOS and two NMOSs, and P_1 is connected to one PMOS and one NMOS. So, it is possible for P_1' to be a candidate for matching P_1 . In another case, N_3' is connected to one PMOS and N_1 is connected to two PMOSs. It is obvious that N_3' is not a candidate for matching N_1 .

Suppose a transistor x' is connected to a transistor y' in T and a transistor x is connected to a transistor y in S . There are three cases for the weights $W_S(x, y)$ and $W_T(x', y')$

The first case is $W_S(x, y) > W_T(x', y')$. In this case, we claim that there is a link between a terminal of x and a terminal of y in S which does not exist in the connection between x' and y' in T . Suppose otherwise, $W_T(x', y')$ will be larger than or equal to $W_S(x, y)$ by Equation (1). This case occurs in Fig. 5. In Fig. 5, there is a link between the gate of transistor x to the gate of transistor y in S , but there is no link between the gate of x' and the gate of y' . We can see that $W_T(x', y') = 8 < W_S(x, y) = 128$. Thus connection (x, y) of S cannot be mapped to connection (x', y') of T .

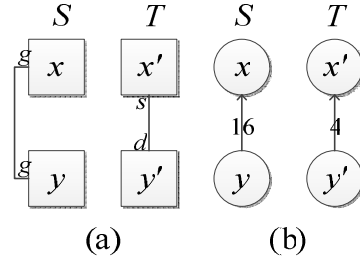


Fig. 5 (a) circuit graphs of circuit S and T (b) WDGs of S and T where $W_S(x, y) > W_T(x', y')$.

We have the following cases:

Case 1: $W_S(x, y) > W_T(x', y')$. The connection (x, y) of S cannot be mapped to connection (x', y') of T .

Case 2: $W_S(x, y) = W_T(x', y')$. The connection (x, y) of S can be mapped to connection (x', y') of T .

Case 3: $W_T(x', y') > W_S(x, y)$. There are two possible cases.

Case 3.1 The types of links for (x, y) are included in the types of links for (x', y') . In this case, connection (x, y) of S can be mapped to connection (x', y') of T .

Case 3.2 The types of links for (x, y) are not included in the types of links for (x', y') . In this case, connection (x, y) of S can not be mapped to connection (x', y') of T .

Consider Fig. 6. Case 3.1 occurs in Fig. 6(a) and Case 3.2 occurs in Fig. 6(b). In both cases, $W_T(x', y') > W_S(x, y)$.

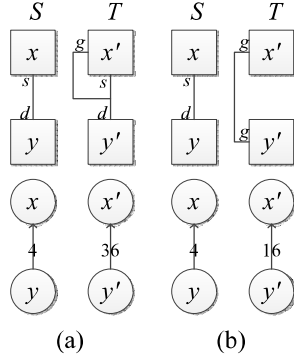


Fig. 6 Two types of connection between two components in T for $W_T(x, y) > W_S(x', y')$

Combining Case 2 and Case 3, we have Rule 2 as follows:

Rule 2: Suppose vertex x is connected to vertex y in WDG of S and vertex x' is connected to vertex y' in WDG of T , where $\tau(x) = \tau(x')$ and $\tau(y) = \tau(y')$. If the edge (x, y) can be mapped to the edge (x', y') , $W_T(x', y')$ must be greater than or equal to $W_S(x, y)$.

We can see that Rule 2 is a filtering rule. We can use it to filter out many edges. In Case 3, although $W_T(x', y') > W_S(x, y)$, we still need to check the connection between the two transistors of T in detail to determine whether we have Case 3.1 or Case 3.2.. We can do this by a bit-parallel approach. Consider Fig. 6. The circuits in Fig. 6(a) belong to Case 3.1 while those in Fig. 6(b) belong to Case 3.2.

For the circuits in Fig. 6(a), we have $W_S(x, y) = [0, 0, 1, 0, 0, 0, 0, 0, 0]$ and $W_T(x', y') = [0, 0, 1, 0, 0, 1, 0, 0, 0]$. In this case, we have $(W_T(x', y') \& W_S(x, y)) = W_S(x, y)$

For the circuits in Fig. 6(b), we have $W_S(x, y) = [0, 0, 1, 0, 0, 0, 0, 0, 0]$ and $W_T(x', y') = [0, 0, 0, 0, 1, 0, 0, 0, 0]$. In this case, we have $(W_T(x', y') \& W_S(x, y)) \neq W_S(x, y)$.

Thus we can use Rule 2.1 to tackle Case 3.

Rule 2.1: Suppose vertex x is connected to vertex y in WDG of S and vertex x' is connected to vertex y' in WDG of T where $\tau(x) = \tau(x')$ and $\tau(y) = \tau(y')$. The edge (x, y) can be mapped to the edge (x', y') , if $(W_T(x', y') \& W_S(x, y)) \neq W_S(x, y)$.

Note that from Rule 2 if $W_T(x', y')$ is smaller than $W_S(x, y)$, we do not use Rule 2.1 again. In the above discussion, we have presented Rules 1 and 2. They are all necessary conditions for x to

be a candidate of x' . We shall now give Rule 3 which is another necessary condition for x to be a candidate of x' . For Rules 1 and 2, our focus is on edges. In other words, we check whether an edge (x, y) can be mapped to another edge (x', y') . In Rule 3, our focus is on vertices.

Rule 3: Let x be a vertex in the WDG of S and x' be a vertex in the WDG of T . If for each edge connected to vertex v from x in WDG of S , there exists at least one edge connected to v' from x' in WDG of T such that (x, v) can be mapped to (x', v') , then x' can be a candidate of x ; otherwise, x' can not be a candidate of x .

Consider Fig. 7(a). For x in S and x' in T , so far as (x, v_1) is concerned, there exists (x', v_1') such that (x, v_1) can be mapped to (x', v_1') by Rules 2.1. Similar argument can be applied to (x, v_2) and (x', v_2') . Therefore x' is a candidate for x . Consider Fig. 7(b), transistor x' cannot be a candidate for x in the circuits in Fig. 7(b) because so far as (x, v_1) is concerned, there does not exist any edge (x', v') such that (x, v_1) can be mapped to (x', v') by Rule 2.

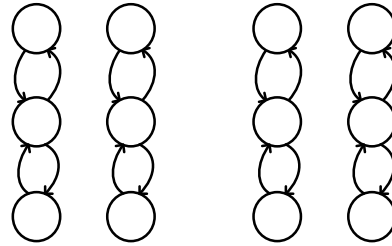


Fig. 7 Examples for Rule 2.1

To implement Rule 3, we first obtain a Boolean vector to be an auxiliary flag vector for the vertex x' in the WDG of T . For a pair of vertices x' in T and x in S , let $F_{x,x'}$ be a n dimensional bit-vector where n is the number of transistors in circuit T . For v_i' in T , $F_{x,x'}(i) = 1$ if and only if there exists a vertex y in S such that (x, y) can be mapped to edge (x', v_i') . Note that if edge (x', v_i') does not exist, $F_{x,x'}(i) = 0$. To determine $F_{x,x'}(i)$, we use Rules 1, 2 and 2.1.

Consider Fig. 8. For $x = v_4$, consider $x' = v_6'$ and $i = 8$. Since (v_4, v_1) can be mapped to (v_6', v_8') by Rule 2.1, $F_{x,x'}(8) = 1$. On the other hand, for $x = v_4$, $x' = v_6'$ and $i = 5$, there is no y in S such that (v_4, y) can be mapped to (v_6', v_5') by Rule 2. Thus, $F_{v_4,v_6'}(5) = 0$. For $i = 2$, since v_2' is not connected to $x' = v_6'$, we do not need to consider it. Therefore, $F_{v_4,v_2'}(2) = 0$.

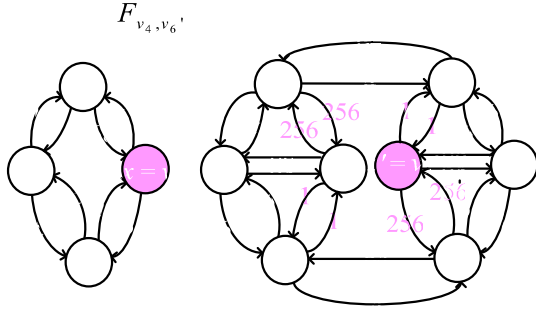


Fig. 8 The auxiliary flag vector for the vertex v_6' in the WDG of T and the vertex v_4 in the WDG of S

We have a further rule to check whether x' in T can be a candidate for x in S . Denote a counter $C_p(x, x')$ and $C_n(x, x')$ to record the total number of *PMOS* and *NMOS* vertices which are mapped to (x', v_i') in S where v_i' is connected to x respectively.

Rule 4 $C_p(x, x') \geq pmos(x')$ and $C_n(x, x') \geq nmos(x')$ if and only if x is a candidate for x' .

Suppose that $\tau(v_i) \in nmos$ and $\tau(v_2) \in pmos$ in S and $\tau(v_8') \in nmos$ and $\tau(v_5'), \tau(v_7') \in pmos$ in T . For $x' = v_6'$ and $x = v_4$, $C_p(x, x') = 0$ by Rule 3. In this case, $C_p(x, x') = 0 < pmos(v_4) = 1$. $x = v_4$ is not a candidate for $x' = v_6'$.

For vertex x in S , we check the edge (x, v_i) where $\tau(v_i) \in pmos$ first. If $C_p(x, x') < pmos(x)$, we do not need to check the edge (x, v_i) where $\tau(v_i) \in nmos$. Let $\delta(x)$ be the candidate set for the vertex x in S such that elements $\{e(x, 1), e(x, 2), \dots, e(x, |\delta(x)|)\}$ of $\delta(x)$ are candidates for x . If $C_p(x, x') \geq pmos(x)$ and $C_n(x, x') \geq nmos(x)$, we add x into the candidate set $\delta(x')$.

Rules 2, 2.1, 3 and 4 are implemented by Algorithm 1 as the follows.

Algorithm 1: Algorithm for Rules 2, 2.1, 3 and 4

Input: a vertex v_k in S , a vertex v_i' in T and $\tau(v_k)$ and $\tau(v_i')$

Output: whether v_k is a candidate of v_i' or not according to Rule 3.

```

 $C_{\tau(v_k)} := 0;$  //  $C_{\tau(v_k)}$  is the number of  $\tau(v_k)$  type transistors connected to  $v_i'$ 
for the neighbor vertex  $v_l$  of  $v_k$  in  $S$ ,  $1 \leq l \leq \beta$  do
    for the neighbor vertex  $v_j'$  of  $v_i'$  in  $T$ ,  $1 \leq j \leq \alpha$  do
         $F_{v_k, v_j'} = 0^n;$ 
        if  $W_T(v_i', v_j') = W_S(v_k, v_l)$  then //Rule 2
            if  $F_{v_k, v_l}(k) = 0$  then //the  $k$ -th edge has not been selected yet
                 $F_{v_k, v_l}(k) := 1;$  //Select the  $k$ -th edge
                 $C_{\tau(v_k)} := C_{\tau(v_k)} + 1;$  //counter is increased by one
            end if
        else if  $W_T(v_i', v_j') > W_S(v_k, v_l)$  then //Rule 2
            if  $((W_T(v_i', v_j') \& W_S(v_k, v_l)) \equiv W_S(v_k, v_l))$  then //Rule 2.1
                if  $F_{v_k, v_l}(k) = 0$  then //the  $k$ -th edge has not been selected yet
                     $F_{v_k, v_l}(k) := 1;$  //Select the  $k$ -th edge
                     $C_{\tau(v_k)} := C_{\tau(v_k)} + 1;$  //counter is increased by one
                end if
            end if
        end for
    end for
if  $\tau(v_i') \in pmos$  and  $F_{v_k, v_i'} \neq 0^n$  and  $C_p < pmos(v_k)$  then //Rule 4
     $\text{Return } \textit{ture};$  //  $v_i'$  is not a candidate of  $v_k$ 
else if  $\tau(v_i') \in nmos$  and  $F_{v_k, v_i'} \neq 0^n$  and  $C_n < nmos(v_k)$  then //Rule 4
     $\text{Return } \textit{ture};$  //  $v_i'$  is not a candidate of  $v_k$ 
else
     $\text{Return } \textit{false};$ 
end if

```

And, Algorithm 2 is the whole bit-parallel filtering algorithm which is shown in the following.

Consider Fig. 10 and Fig. 11. For v_1 in S , vertices v_3' , v_4' , v_7' , v_8' , v_9' and v_{11}' of T are filtered out by transistor type. And, vertices v_1' , v_5' , v_6' , v_8' and v_{10}' are filtered out by Rule 3. Thus we have that v_2' is a candidate of v_1 . For v_4 in S , vertices v_1' , v_2' , v_5' , v_6' and v_{10}' are filtered by transistor type, and vertices v_4' , v_8' and v_9' of T are filtered out by Rule 1. Thus v_3' , v_7' and v_{11}' are candidates of v_4 .

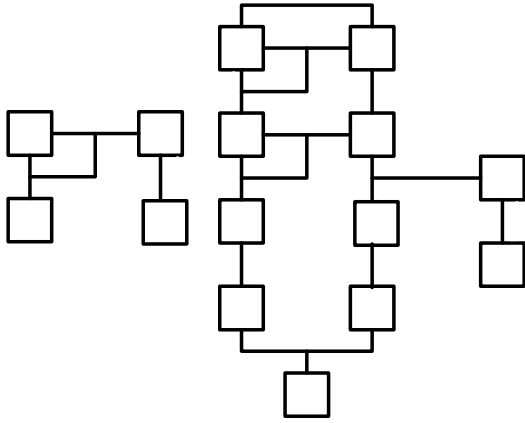


Fig. 10 (a) Subcircuit S (b) Circuit of Operational Amplifier T

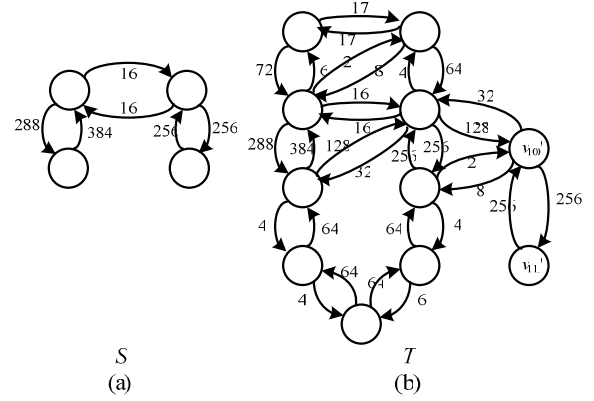


Fig. 11 The WDG of Fig. 10. (a)

$\tau(v_1), \tau(v_3) \in pmos$ and $\tau(v_2), \tau(v_4) \in nmos$ (b)

$\tau(v_1'), \tau(v_2'), \tau(v_5'), \tau(v_6'), \tau(v_{10}') \in pmos$ and
 $\tau(v_3'), \tau(v_4'), \tau(v_7'), \tau(v_8'), \tau(v_9'), \tau(v_{11}') \in nmos$.

After filtering out nodes in T which are not candidates of any node in S , we obtain the following S and T' as shown in Fig. 12. Note that there exists a one-to-one corresponding for the edges in two graphs.

Algorithm 2: The Bit-Parallel Filtering for Subcircuit Extraction Algorithm

Input : The text circuit T and the pattern circuit S

Output : The candidate set $\delta(v)$ of S for all v

// setup step

Construct the weighted directed graph $G_T = (V_T, E_T)$ of the circuits T where $V_T = \{v_1', v_2', \dots, v_n'\}$ and the weighted directed graph $G_S = (V_S, E_S)$ of the circuits S where $V_S = \{v_1, v_2, \dots, v_m\}$;

// filtering step

for vertex v_k , $1 \leq k \leq n$ **do**

for vertex v_i' , $1 \leq i \leq m$ **do**

if $nmos(v_i') \geq nmos(v_k)$, $pmos(v_i') \geq pmos(v_k)$ and $\tau(v_i') = \tau(v_k)$ **then**

 // Rule 1

$F := 0^n$; $C_p := 0$; $C_n := 0$;

If Algorithm 1 with inputs v_k , v_i' and $\tau(v_k')$ returns *true* **then**

 break; // v_i' is not a candidate of v_k by Algorithm 1

else

 add v_i' into $\delta(v_k)$;

 // v_i' is a candidate of v_k

end if

end if

end for

end for

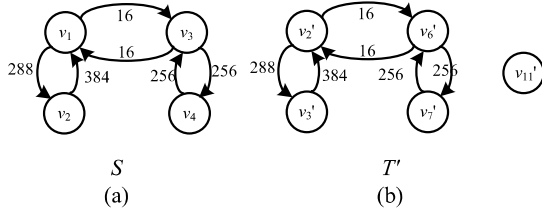


Fig. 12 (a) Circuit S (b) The remaining graph T' of T

4 The Algorithm to Find All Occurrences of S in T

In the above sections, we discussed for a given node x in circuit S , how to filter out all of the nodes in T which cannot be candidates of x . In this section, we shall give our algorithm to examine the entire circuit of S to see whether it appears in T . This can be done by a Breadth-First search (BFS).

Consider Fig. 10. Suppose we select v_1 as the starting node in S . We conduct a BFS, and we will have the graph as shown in Fig. 13(a). As we pointed out before, the candidate set of v_1 is $\{v_2'\}$. We then consider v_2 which is a descendent of v_1 . For v_2 , its candidate set is $\{v_3'\}$. We check whether v_2' is connected to v_3' in T' . We continue this process until we get Fig. 13(b). Fig. 13 shows that for circuit S there exists a circuit T' in T . In this case, T' is identical to S .

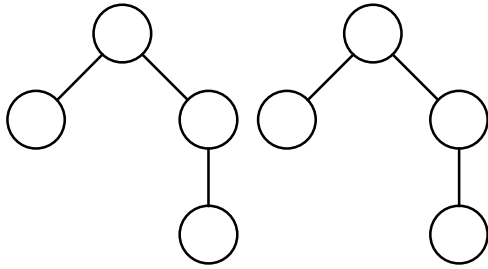


Fig. 13 (a) A BFS for circuit S in Fig. 10(a) (b) the candidate set is built by the BFS of circuit S .

Although for circuit S , we can obtain a corresponding circuit T' in T by BFS approach, we have to do a further checking. Consider Fig. 14.

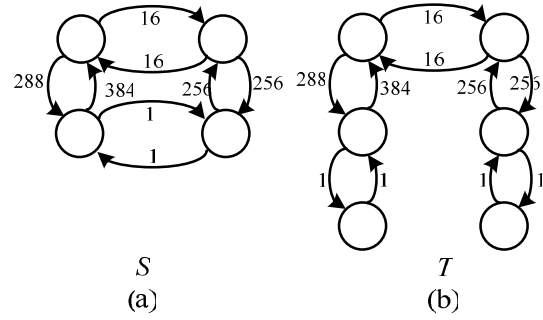


Fig. 14 Circuits S and T for another case

In this case, after filtering process, we have $\delta(v_1) = \{v_1'\}$, $\delta(v_2) = \{v_2'\}$, $\delta(v_3) = \{v_4'\}$ and $\delta(v_4) = \{v_5'\}$. If v_1 is selected as a starting node for BFS searching, we also conduct the same solution as Fig. 13(a). However, this solution is a wrong one, because v_2 is connected to v_4 in S while v_2' is not connected to v_5' in T .

We use connection matrices for both circuits S and subcircuits of T for the solution to make sure that it is a correct answer. The definition of the connection matrix is shown in the following.

Suppose that a graph G with n vertices. The connection matrix M_G is a Boolean matrix with size $n \times n$. $M_G(x, y)$ is 0 if there is no edge between vertices x and y in G ; otherwise, $M_G(x, y)$ is 1.

We shall use Rule 5 as the confirmation step for the candidate set.

Rule 5: Let $S = \{v_1, v_2, \dots, v_n\}$ and $T' = \{v_1', v_2', \dots, v_n'\}$ in T where $x_i' \in \delta(x_i)$ for $1 \leq i \leq n$. Let $M_G(i) = [M_G(i, 1), M_G(i, 2), \dots, M_G(i, n)]$. If S exactly matches T' , $((M_S(i) \& M_{T'}(i)) \equiv M_S(i))$ is true for all $1 \leq i \leq n$; otherwise, S does not match T' .

Consider the circuits in Fig. 14. The two matrices M_S and $M_{T'}$ are shown in Fig. 15. We can see that $M_S(2) \& M_{T'}(2) = [1, 0, 0, 0] \neq M_S(2) = [1, 0, 0, 1]$. This shows that the two circuits are not identical.

Algorithm 3: BFS Algorithm $bfs(level)$

```

if  $level > m$  then
    if Rule 5 is true then
        Report the matching set;
    end if
else
    for  $i = 1$  to  $|\delta(q_{level})|$  do
    
```

```

If  $e(q_{level}, i)$  is not selected then
    add  $e(q_{level}, i)$  into the matching set;
    If  $e(q_{level}, i)$  is connected to
     $parent(q_{level})$  then
         $bfs(level+1)$ ;
    end if
    remove  $e(q_{level}, i)$  from the
    matching set;
end if
end for
end if

```

5 Experiments

We implemented the proposed approach in this section. The computer used in the tests was a 600Mhz Pentium 3 with 1G RAM and Linux OS. All code was compiled with GCC. In Experiment 1, the tests involved target circuits of length 1k, 2k, ..., 9k and 10k with the NAND circuit of length 4. The result is shown in Fig. 15.

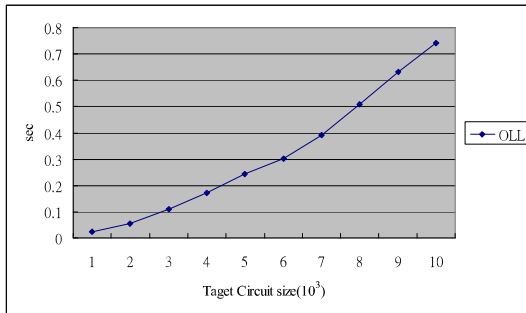


Fig. 15 Experiment 1

In Experiment 2, the tests involved pattern circuits of length 10, 20, 30, 40 and 50 with the target circuit of length 10k. In this experiment, we did not consider the time transformed HSPICE of text and pattern into the WDG. The result is shown in Fig. 16.

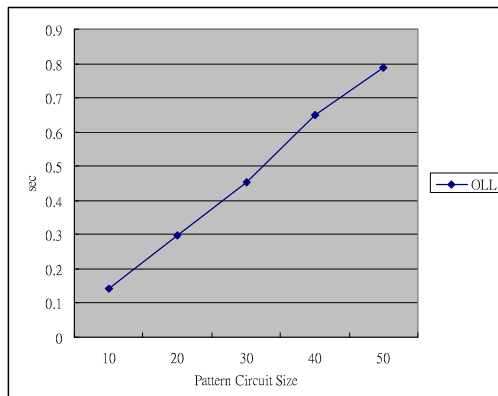


Fig. 16 Experiment 2

6 Conclusion

In this paper, we proposed a new structure to represent the circuit net and use a bit-parallel algorithm to achieve the filtering algorithm. Our bit-parallel algorithm is efficient and useful to filter out impossible circuits from main circuit. Our algorithm also can allow containing other devices like capacitor, inductor, resistor, VDD, GND and so on. And, according to results of our experiments, we also can conclude that our algorithm is efficient.

References

- [1] W. H. Chang, S. D. Tzeng, and C. Y. Lee, "Novel Subcircuit Extraction Algorithm by Recursive Identification Scheme," *IEEE International Symposium on Circuits and Systems*, Vol. 5, pp. 491-494, 2001.
- [2] M. Ohlrich, C. Ebeling, E. Ginting, and L. Sather, "SubGemini: Identifying SubCircuits using a Fast Subgraph Isomorphism Algorithm," *Proceedings of Design Automation Conference*, pp. 31-37, June 1993.
- [3] N. Vijaykrishnan and N. Ranganathan, "SUBGEN: A Genetic Approach for Subcircuit Extraction," *9th International Conference on VLSI Design*, pp. 343-345, January 1996.
- [4] N. Zhang and D. C. Wunsch II, "A Novel Subcircuit Extraction Algorithm using Heuristic Dynamic Programming (HDP)," *2002 International Conference on VLSI, Las Vegas, Nevada*, June 24-27, 2002.
- [5] N. Zhang and D. C. Wunsch II, "Speeding up VLSI Layout Verification Using Fuzzy Attributed Graphs Approach," *IEEE Transactions on Fuzzy Systems*, Vol. 14, pp. 728-737, 2006.
- [6] N. Zhang, D. C. Wunsch II, and F. Haray, "The Subcircuit Extraction Problem," *IEEE Potentials*, Vol. 22, pp. 22-25, 2003.